Documentação e Relatório de Testes da API (Etapa 3)

3.1. Integração do Swagger

A ferramenta Swagger UI foi integrada ao pipeline do ASP.NET Core utilizando a biblioteca padrão Swashbuckle.AspNetCore. A interface gerada permite a visualização interativa do contrato da API (OpenAPI) e a execução de testes manuais diretamente contra as tabelas do banco de dados SQL Express, garantindo o acompanhamento em tempo real das persistências.

3.2. Documentação dos Endpoints (Métodos, Parâmetros e Respostas)

A documentação estrutural foi injetada no código-fonte através de Data Annotations, evitando redundâncias em arquivos externos. O padrão adotado para todos os controladores da API é:

[GET] Listagem Geral (/api/{entidade}):

Parâmetros: Nenhum.

Resposta: 200 OK (Retorna a coleção de entidades em formato JSON).

[GET] Busca por Chave Primária (/api/{entidade}/{id}):

Parâmetros: ID (Inteiro) injetado na URL.

Resposta: 200 OK (Objeto JSON) ou 404 Not Found.

[POST] Criação de Registro (/api/{entidade}):

Parâmetros: Objeto JSON (application/json) no Body da requisição.

Resposta: 201 Created (Sucesso) ou 400 Bad Request (Falha nas regras de negócio).

[PUT] Atualização (/api/{entidade}/{id}):

Parâmetros: ID numérico na URL e objeto JSON atualizado no Body.

Resposta: 204 No Content (Sucesso na alteração) ou 400 Bad Request.

[DELETE] Exclusão (/api/{entidade}/{id}):

Parâmetros: ID numérico na URL.

Resposta: 204 No Content (Sucesso na exclusão) ou 404 Not Found.

3.3. Testes Manuais de Funcionalidade (Validação CRUD)

Os testes de integração foram executados utilizando a interface do Swagger, simulando o ciclo de vida completo de um registro na tabela Veiculos.

A. Inserção (Create)

O payload enviado respeitou as chaves estrangeiras. O Entity Framework gerou o ID automaticamente.

Curl

```
curl -X 'POST' \
  'https://localhost:7029/api/Veiculos' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
  "modelo": "Pulse",
  "anoFabricacao": 2025,
  "quilometragem": 0,
  "fabricanteId": 1,
  "categoriaId": 1
}'
```

Request URL

```
https://localhost:7029/api/Veiculos
```

Server response

Code	Details
201	Response body

```
{
  "id": 3,
  "modelo": "Pulse",
  "anoFabricacao": 2025,
  "quilometragem": 0,
  "fabricanteId": 1,
  "fabricante": null,
  "categoriaId": 1,
  "categoria": null
}
```

Download

B. Leitura ID (Read)

O recurso foi recuperado através de seu ID identificador, trazendo também as entidades aninhadas (Join implícito do EF).

Name	Description
id * required integer($int32) (path)	1

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7029/api/Veiculos/1' \
  -H 'accept: text/plain'
```

Request URL

```
https://localhost:7029/api/Veiculos/1
```

Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "modelo": "Argo Drive",
  "anoFabricacao": 2024,
  "quilometragem": 10000,
  "fabricanteId": 1,
  "fabricante": null,
  "categoriaId": 1,
  "categoria": null
}
```

Download

B. Leitura Sem ID

A API retornou o Status Code 200 OK. A coleção completa de recursos foi recuperada, retornando todos os registros da base de dados acompanhados de suas respectivas entidades aninhadas (Join implícito do EF).

Code	Details
200	Response body


```
[
  {
    "id": 1,
    "modelo": "Argo Drive",
    "anoFabricacao": 2024,
    "quilometragem": 10000,
    "fabricanteId": 1,
    "fabricante": {
      "id": 1,
      "nome": "Fiat",
      "veiculos": [
        null,
        {
          "id": 3,
          "modelo": "Pulse",
          "anoFabricacao": 2025,
          "quilometragem": 0,
          "fabricanteId": 1,
          "fabricante": null,
          "categoriaId": 1,
          "categoria": {
            "id": 1,
            "nome": "Hatch",
            "valorDiariaBase": 120.5,
            "veiculos": [
              null,
              null
            ]
```


Download

C. Modificação (Update)

O estado da entidade foi modificado com sucesso, confirmando a transação via método PUT.

```
{
  "id": 1,
  "modelo": "Argo Drive Atualizado",
  "anoFabricacao": 2024,
  "quilometragem": 50000,
  "fabricanteId": 1,
  "categoriaId": 1
}
```

Execute | Clear

Responses

Curl

```
curl -X 'PUT' \
  'https://localhost:7029/api/Veiculos/1' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 1,
  "modelo": "Argo Drive Atualizado",
  "anoFabricacao": 2024,
  "quilometragem": 50000,
  "fabricanteId": 1,
  "categoriaId": 1
}'
```

Request URL

```
https://localhost:7029/api/Veiculos/1
```

Server response

Code	Details
204	Response headers

```
date: Mon,13 Apr 2026 00:36:40 GMT
server: Kestrel
```

D. Remoção (Delete)

A instrução de exclusão localizou a entidade e a retirou fisicamente da base de dados.

Name	Description
id * required integer($int32) (path)	3

Execute | Clear

Responses

Curl

```
curl -X 'DELETE' \
  'https://localhost:7029/api/Veiculos/3' \
  -H 'accept: */*'
```

Request URL

```
https://localhost:7029/api/Veiculos/3
```

Server response

Code	Details
204	Response headers

```
date: Mon,13 Apr 2026 00:38:47 GMT
server: Kestrel
```

3.4. Testes de Robustez e Qualidade (Validação)

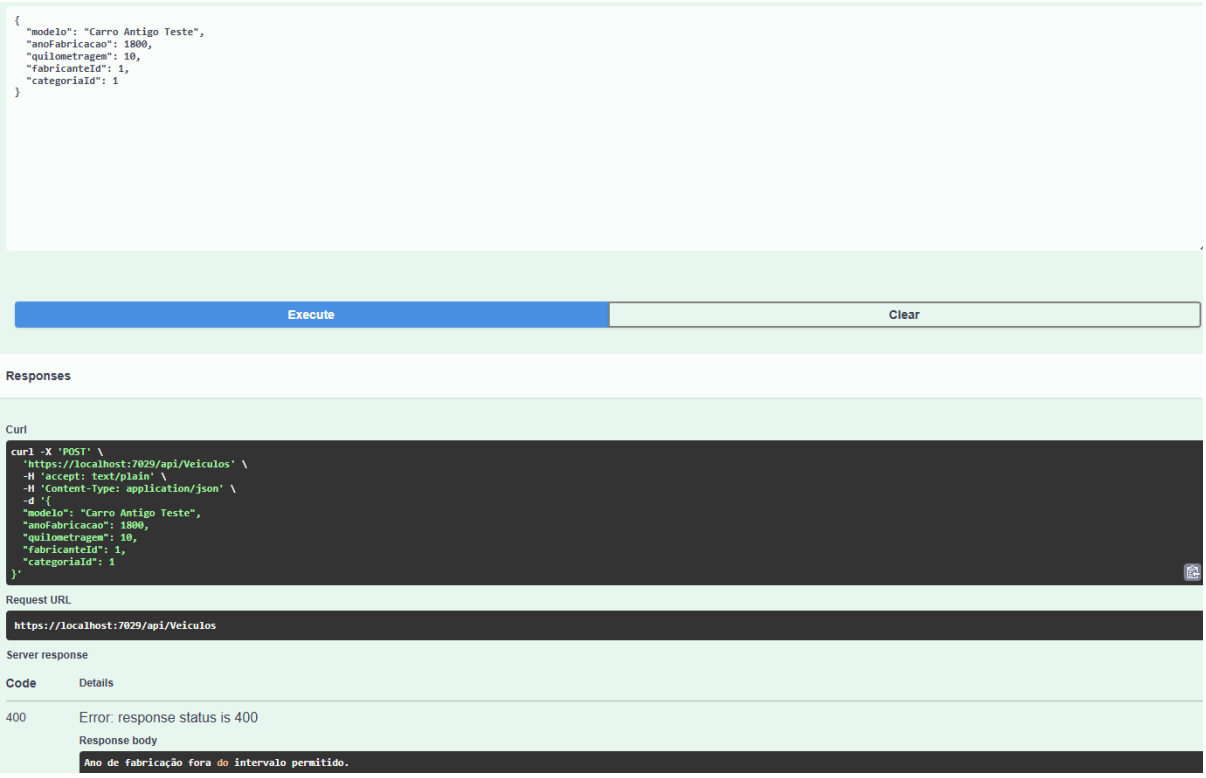
Para garantir a confiabilidade da aplicação e blindar o banco de dados contra a corrupção de informações, o backend foi submetido a diversos cenários de estresse e inserção de dados inconsistentes. A API demonstrou resiliência, interceptando as anomalias nas camadas de controle antes de qualquer transação com o Entity Framework.

Abaixo estão as evidências dos testes de regras de negócio e validações:

Exemplo 1: Validação de Limite Numérico (Veículos)

Cenário: Requisição POST na rota de Veículos enviando o atributo anoFabricacao intencionalmente fora do limite estipulado pelo negócio (Ano: 1800).

Resultado: O sistema abortou a operação e retornou o Status Code 400 Bad Request, protegendo a base de dados contra datas irreais.



Exemplo 2: Validação de Regra Temporal (Aluguéis)

Cenário: Tentativa de inserção (POST) de um novo contrato de locação onde a DataDevolucao cronologicamente antecede a DataInicio.

Resultado: O controlador identificou a inconsistência temporal e bloqueou a transação, emitindo o Status Code 400 Bad Request com uma mensagem clara sobre a violação da regra de negócio.

```
  "dataInicio": "2026-04-20T10:00:00",
  "dataDevolucao": "2026-04-15T10:00:00",
  "quilometragemInicial": 10000,
  "quilometragemFinal": 10500,
  "valorDiaria": 120.00,
  "valorTotal": 600.00,
  "clienteId": 1,
  "veiculoId": 1
}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7029/api/Alugueis' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
  "dataInicio": "2026-04-20T10:00:00",
  "dataDevolucao": "2026-04-15T10:00:00",
  "quilometragemInicial": 10000,
  "quilometragemFinal": 10500,
  "valorDiaria": 120.00,
  "valorTotal": 600.00,
  "clienteId": 1,
  "veiculoId": 1
}'
```

Request URL

```
https://localhost:7029/api/Alugueis
```

Server response

Code	Details
400	Error: response status is 400 Response body `A data de devolução não pode ser anterior à data de início.`

Exemplo 3: Validação de Campos Obrigatórios (Clientes)

Cenário: Requisição POST enviando o payload do cliente com a string do CPF vazia ou contendo apenas espaços em branco (IsNullOrWhiteSpace).

Resultado: Rejeição da entrada com Status Code 400 Bad Request, atestando que os dados mínimos exigidos pelo sistema estão sendo validados.

```
{
  "nome": "João Silva",
  "cpf": "",
  "email": "joao@email.com"
}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7029/api/Clientes' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
  "nome": "João Silva",
  "cpf": "",
  "email": "joao@email.com"
}'
```

Request URL

```
https://localhost:7029/api/Clientes
```

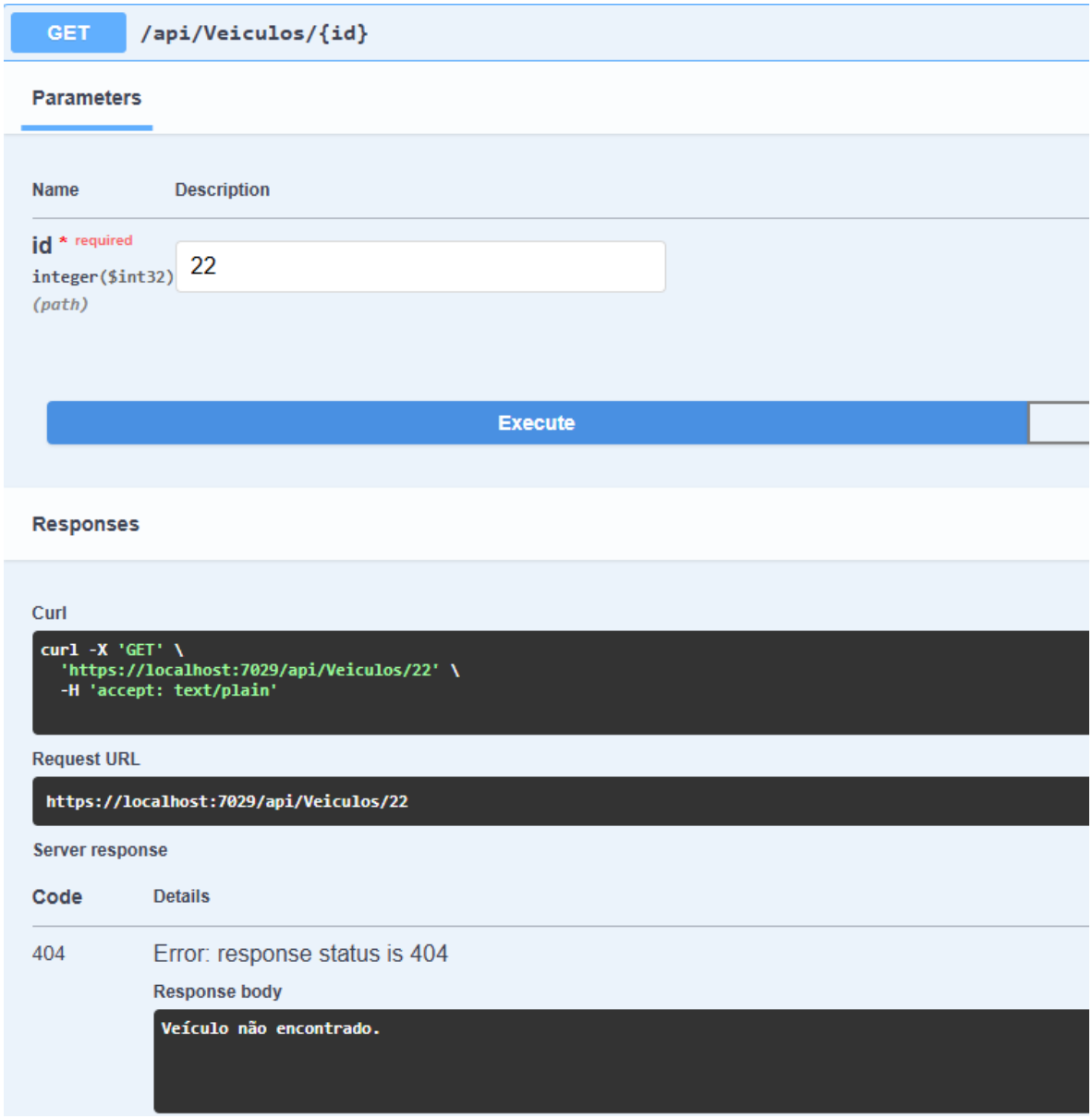
Server response

Code	Details
400	Error: response status is 400 Response body `O CPF é obrigatório.`

Exemplo 4: Tratamento de Rotas Inexistentes (Not Found)

Cenário: Requisição de leitura (GET) apontando para um ID de recurso que não está cadastrado na base de dados (Ex: ID 22).

Resultado: Em vez de gerar uma falha interna no servidor de banco de dados, a API tratou a ausência graciosamente, retornando o Status Code 404 Not Found, conforme as melhores práticas do padrão RESTful.



3.5. Documentação e Testes dos Filtros Produzidos (Relatórios)

O sistema de relatórios foi projetado para extrair informações estratégicas do banco de dados utilizando consultas LINQ otimizadas. A estrutura

dispensa o uso de laços de repetição em memória, delegando o processamento dos relacionamentos diretamente para o mecanismo do SQL Server.

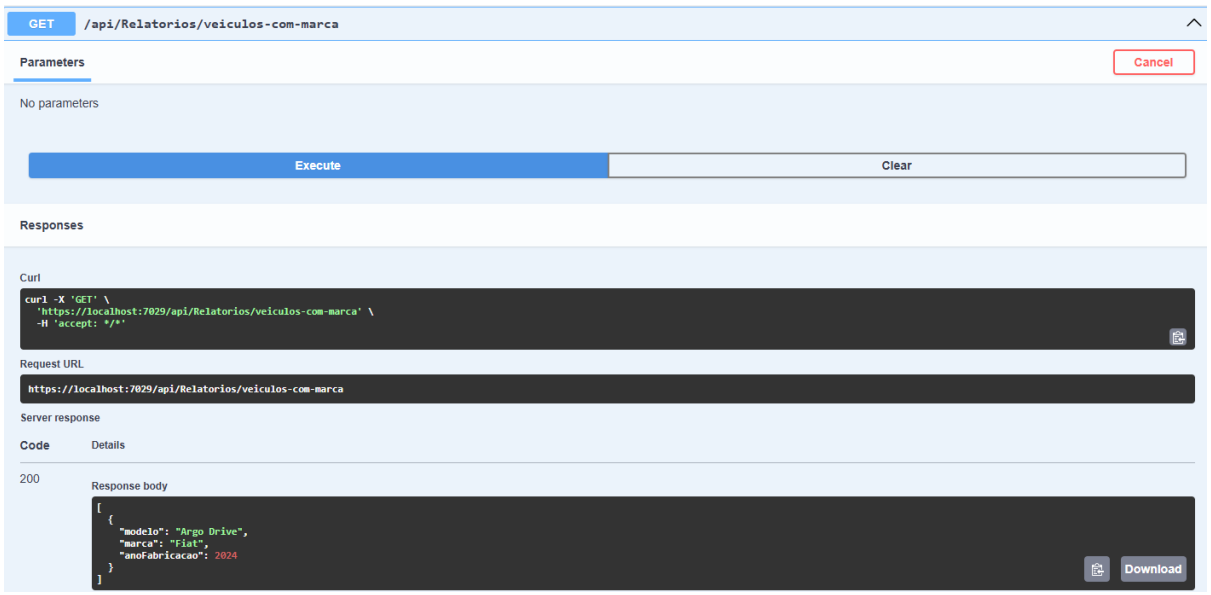
Abaixo estão as execuções das 5 rotas de relatórios, acompanhadas de suas respectivas descrições técnicas e evidências de funcionamento.

Consulta 1: Catálogo de Veículos com Marca

Rota Swagger: GET /api/Relatorios/veiculos-com-marca

Descrição do Filtro: Utiliza a instrução estrutural equivalente a um Inner Join. A consulta cruza as chaves entre as tabelas de Veiculos e Fabricantes, substituindo o identificador numérico (FabricanteId) pelo nome real e descritivo da marca, facilitando a leitura pelo usuário final.

Evidência de Execução:

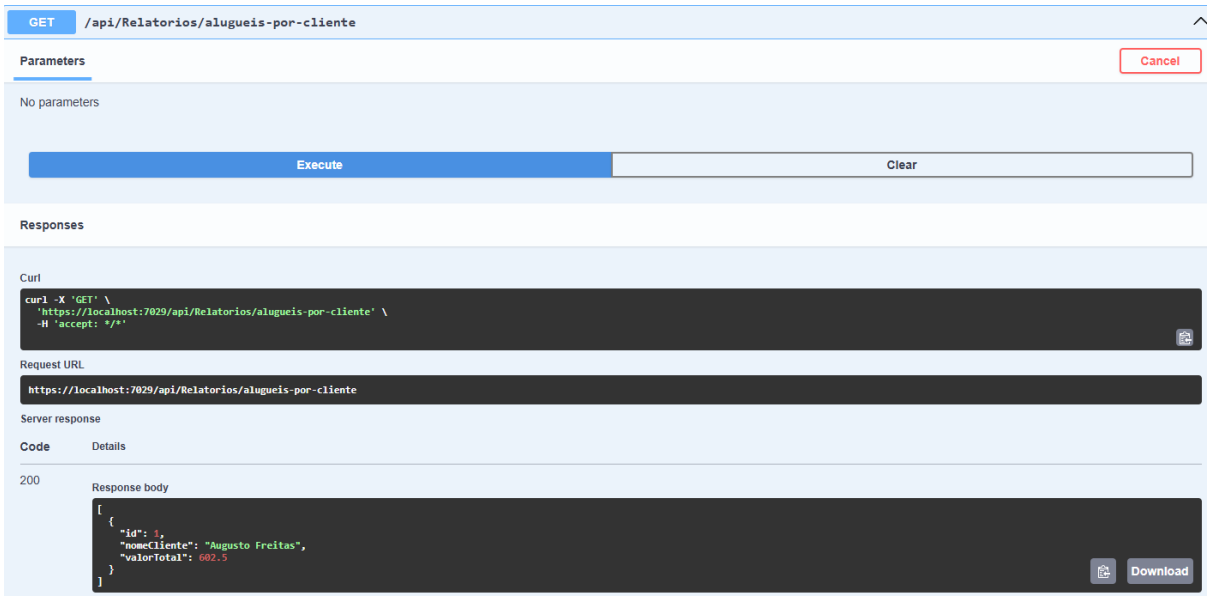


Consulta 2: Responsabilidade de Aluguéis

Rota Swagger: GET /api/Relatorios/alugueis-por-cliente

Descrição do Filtro: Aplica um relacionamento direto (Inner Join) entre a tabela de transações (Alugueis) e a tabela de cadastro (Clientes). O objetivo é vincular os dados financeiros de um contrato ativo diretamente ao nome do locatário responsável pela transação.

Evidência de Execução:

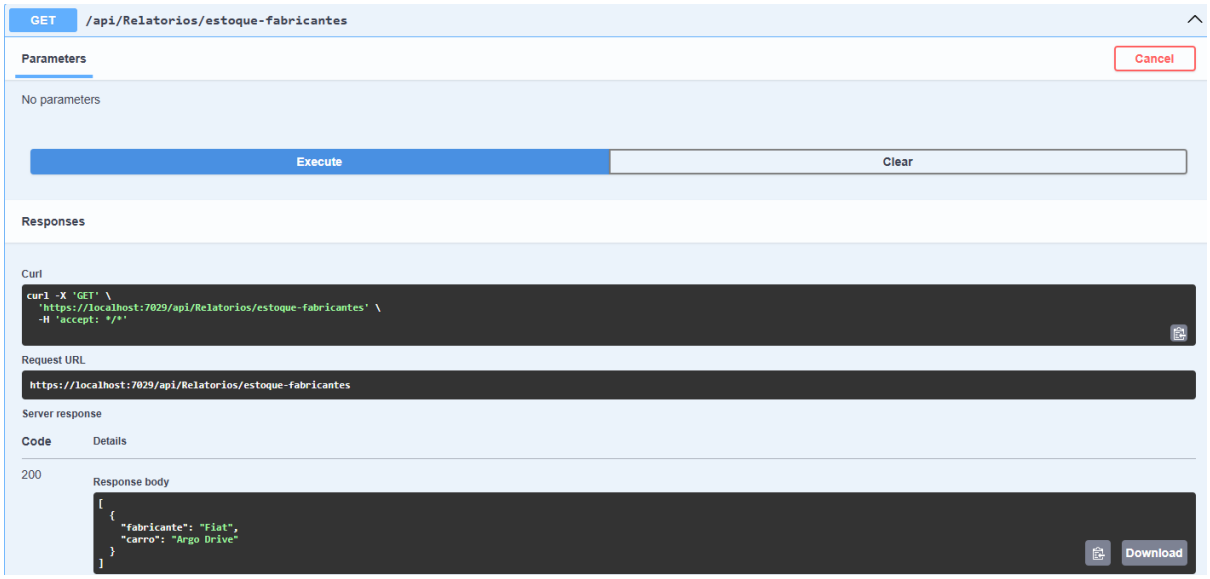


Consulta 3: Visão Geral de Estoque

Rota Swagger: GET /api/Relatorios/estoque-fabricantes

Descrição do Filtro: Implementa uma junção externa esquerda (Left Outer Join) utilizando o método .DefaultIfEmpty(). Trata-se de um filtro estratégico que lista todas as marcas de fabricantes do banco de dados, assegurando que montadoras recém-cadastradas que ainda não possuem veículos vinculados apareçam no relatório com a sinalização "Sem Estoque".

Evidência de Execução:

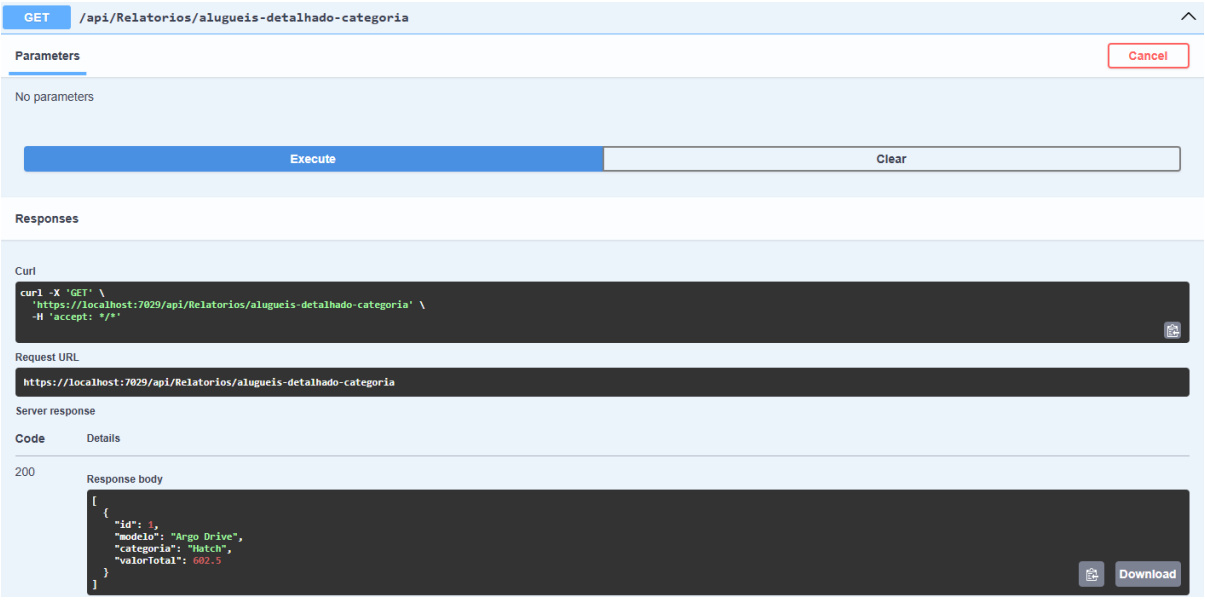


Consulta 4: Desempenho por Categoria

Rota Swagger: GET /api/Relatorios/alugueis-detalhado-categoria

Descrição do Filtro: Executa um encadeamento relacional mais complexo, conhecido como Join Triplo. A consulta parte da tabela de Alugueis, conecta-se à tabela de Veiculos para identificar o bem locado, e faz um terceiro cruzamento com a tabela de Categorias. Isso permite detalhar a origem da locação (ex: descobrir que um aluguel específico foi de um carro da categoria "SUV").

Evidência de Execução:



Consulta 5: Ranking Financeiro Consolidado

Rota Swagger: GET /api/Relatorios/faturamento-total-clientes

Descrição do Filtro: Trata-se de uma projeção analítica. Além de realizar o Inner Join entre Alugueis e Clientes, a instrução LINQ acopla a cláusula Group By para agrupar as transações por cliente. Em seguida, aplica a função de agregação matemática .Sum() na coluna de valores, gerando o extrato do faturamento total que cada cliente já gerou para a locadora.

Evidência de Execução:

GET /api/Relatorios/faturamento-total-clientes

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7029/api/Relatorios/faturamento-total-clientes' \
  -H 'accept: */*'
```

Request URL

```
https://localhost:7029/api/Relatorios/faturamento-total-clientes
```

Server response

Code	Details
200	Response body


```
[
  {
    "cliente": "Augusto Freitas",
    "totalGasto": 602.5
  }
]
```


Download